



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temario para la preparación de la Oposición al Cuerpo Superior de
Sistemas y Tecnologías de la Información de la Administración del
Estado.

TEMAS ESPECÍFICOS – BLOQUE B

I. Organización y gestión de los sistemas de información

**Tema 126. La seguridad en el nivel de aplicación. Tipos de
ataques y protección de servicios web, bases de datos e
interfaces de usuario.**

AUTORA: María Riesco García.

REVISOR: Leticia Merino Cancela.

Creación: septiembre 2023
Actualización:

ÍNDICE

1.	LA SEGURIDAD EN EL NIVEL DE APLICACIÓN.	4
2.	TIPOS DE ATAQUES DE APLICACIONES Y ENTORNOS WEB.	5
2.1	INTRODUCCIÓN.....	5
2.2	RIESGOS Y AMENAZAS DE SEGURIDAD EN ENTORNOS WEB	5
2.3	RIESGOS Y AMENAZAS DE SEGURIDAD EN ENTORNOS WEB SEGÚN OWASP ...	6
2.4	CONCLUSIONES	8
3.	ESTRATEGIA Y METODOLOGÍA DE SEGURIDAD DE APLICACIONES WEB. PROTECCIÓN DE ENTORNOS WEB.	9
3.1	INTRODUCCIÓN.....	9
3.2	SEGURIDAD EN ENTORNOS WEB.....	9
3.3	ARQUITECTURAS DE SEGURIDAD EN ENTORNOS WEB	10
3.4	CONCLUSIÓN	11
4.	DESARROLLO SEGURO DEL SOFTWARE DE APLICACIONES	12
4.1	INTRODUCCIÓN.....	12
4.2	RECOMENDACIONES GENERALES	12
4.2.1	FILTRADO DE DATOS DE ENTRADA DEL USUARIO	12
4.2.2	MENSAJES DE ERROR Y OTROS CONTENIDOS	13
4.2.3	AUTENTIFICACIÓN Y GESTIÓN DE SESIONES	13
4.2.4	CSRF.....	13
4.2.5	GESTIÓN DE LOGS	14
4.3	CONCLUSIÓN	14
5.	ANÁLISIS DE SEGURIDAD DE APLICACIONES WEB	15
5.1	INTRODUCCIÓN.....	15
5.2	METODOLOGÍA DE ANÁLISIS DE CAJA NEGRA	15
5.3	METODOLOGÍA DE ANÁLISIS DE CAJA BLANCA	15
5.4	CONCLUSIÓN	15
6.	RESUMEN ESQUEMÁTICO	16
7.	GLOSARIO	17
8.	TEST	18
A)	PREGUNTAS DE TEST	18

B)	SOLUCIONES A LAS PREGUNTAS DE TEST	19
9.	BIBLIOGRAFÍA	20
A.	BIBLIOGRAFÍA BÁSICA	20
B.	BIBLIOGRAFÍA PARA AMPLIAR EL TEMA.....	20

1. La seguridad en el nivel de aplicación.

Para hacer frente a amenazas de ciberseguridad asociadas a las aplicaciones y entornos web, cada vez más abiertos y modulares, se debe contemplar su protección durante todo su ciclo de vida. Es decir, durante el proceso de diseño y desarrollo de la aplicación, y posteriormente, tras la puesta en producción. Durante la parte de desarrollo se establecerán requisitos y verificaciones de seguridad, y cuando se encuentre en producción se realizarán auditorías de seguridad, análisis de vulnerabilidades y pruebas de intrusión.

Con respecto al nivel de aplicación (TCP/IP), los protocolos más utilizados son los siguientes:

- Domain Name Service (DNS) para resolver los nombres de dominio en direcciones IP.
- Simple Mail Transfer Protocol (SMTP) para enviar correo electrónico.
- HyperText Transfer Protocol (HTTP) para navegar por la web.

Las vulnerabilidades de estos protocolos se explotan para afectar a las comunicaciones basadas en ellos.

2. Tipos de ataques de aplicaciones y entornos web.

2.1 Introducción

Las aplicaciones web cuentan con características específicas que de por sí les hacen vulnerables frente a determinados riesgos y amenazas, los cuales se detallan a continuación.

2.2 Riesgos y amenazas de seguridad en entornos web

La seguridad en el proceso de diseño y desarrollo de una aplicación Web no puede abordar solamente el entorno de desarrollo, que deberá ser la última versión con los parches de seguridad actualizados, sino que debe abordar la aplicación web que se desarrolle, con código propio y teniendo en cuenta el lenguaje de programación empleado.

De acuerdo con la Guía de Seguridad en entornos y aplicaciones web CCN-STIC-812, las grandes amenazas de seguridad de las aplicaciones web se dan por las siguientes razones, principalmente:

- “a) Las aplicaciones Web en Internet están públicamente disponibles.*
- b) La ubicuidad de las aplicaciones Web es muy elevada, ya que están disponibles en prácticamente cualquier entorno de computación.*
- c) Las aplicaciones Web estándar utilizan el puerto TCP/80 (HTTP) y las que emplean cifrado mediante SSL (Secure Socker Layer) o TLS (Transport Layer Security) emplean el puerto TCP/443 (HTTPS). Por tanto, los firewalls tradicionales de manera general deben dejar pasar el tráfico hacia estos puertos, y son de poca utilidad en el filtrado de ataques directos sobre la aplicación Web.*
- d) HTTP es un protocolo complejo que permite recibir datos del usuario (en la URL mediante el método GET, en el cuerpo de la petición mediante el método POST, mediante métodos HTTP más avanzados, a través de cookies, mediante cabeceras HTTP, etc.) para su procesamiento.*
- e) Las técnicas y herramientas de ataque necesarias para explotar vulnerabilidades en los entornos Web son muy sencillas, siendo en algunos casos más que suficiente el uso exclusivo de un navegador Web estándar.*
- f) Los ataques desde Internet sobre las aplicaciones Web disponibles públicamente conllevan un anonimato muy elevado por parte del atacante. Este anonimato se incrementa notablemente cuando el atacante emplea dispositivos intermedios (proxies anónimos o VPNs) para ocultar su origen real.*
- g) La mayoría de aplicaciones Web tienen carácter propietario y han sido desarrolladas internamente o mediante la solicitud de este servicio a una tercera compañía. Por estos motivos, es muy probable que la aplicación no haya pasado los controles y verificaciones de seguridad pertinentes para detectar posibles vulnerabilidades de seguridad.*
- h) Las capacidades de autenticación y mantenimiento de sesiones en HTTP son muy limitadas, motivo por el que es necesario mediante software proporcionar un sistema de autenticación y de gestión de sesiones robusto.”*

El desarrollo de una aplicación web, dada su complejidad, puede introducir nuevas vulnerabilidades de seguridad.

Además, no sólo la arquitectura web es objetivo de ataques, sino las comunicaciones entre ellas (el tráfico web y sus conexiones con los usuarios).

Las vulnerabilidades de seguridad más comunes y relevantes en aplicaciones Web en los últimos años son, conforme a la Guía de Seguridad en entornos y aplicaciones web CCN-STIC-812:

- “a) Cross-Site Scripting (XSS): la aplicación Web envía los datos proporcionados por el usuario al navegador Web sin realizar ninguna validación o codificación del contenido. Este ataque*

permite a un atacante ejecutar código (scripts) en el navegador de la víctima: robo de sesiones, modificación de los contenidos de la Web y su configuración.

b) Ataques de inyección: los datos proporcionados por el usuario se envían a un intérprete como parte de un comando o consulta. El ejemplo más conocido es la inyección SQL en bases de datos. El atacante puede ejecutar código dañino y modificar datos a través del intérprete atacado: SQL, XPath, LDAP, etc.

c) Ejecución de ficheros y código malicioso: el código vulnerable a la inclusión de ficheros remotos (o código remoto), Remote File Inclusion (RFI), permite a un atacante incluir código y datos dañinos en la aplicación Web. El resultado permite tener control completo del servidor Web. Este ataque afecta a cualquier entorno de desarrollo (eg. PHP) que acepte ficheros o nombres de ficheros de los usuarios.

d) Referencias directas a objetos inseguras: una referencia directa a un objeto ocurre si el desarrollador expone un objeto interno de la implementación (fichero, directorio, registro de la BD, clave...) en forma de URL o de parámetro de un formulario. Un atacante puede manipular estas referencias para acceder a otros objetos sin autorización.

e) Cross Site Request Forgery (CSRF): los ataques CSRF fuerzan al navegador de la víctima a enviar peticiones pre-autenticadas a una aplicación Web vulnerable. La petición obliga al navegador de la víctima a realizar acciones hostiles no deseadas en beneficio del atacante sobre una sesión previamente establecida. Los límites de este ataque están en el tipo de aplicación Web atacada.

f) Filtrado de información y gestión incorrecta de errores: las aplicaciones Web pueden revelar información sobre su configuración, detalles internos de implementación o violar la privacidad de los datos, de forma no intencionada. El atacante emplea esta debilidad para obtener información sensible o realizar ataques más avanzados.

g) Autenticación y gestión de sesiones: las credenciales de acceso y los tokens (o identificadores) de sesión no son protegidos adecuadamente. Un atacante puede comprometer claves, secretos e identificadores de autenticación y robar la identidad de otros usuarios.

h) Almacenamiento criptográfico inseguro: la aplicación Web no utiliza funciones criptográficas adecuadamente para proteger los datos y las credenciales empleadas. Un atacante puede usar los datos no protegidos para robar la identidad de otros usuarios y realizar otros ataques, como fraude con tarjetas de crédito.

i) Comunicaciones inseguras: la aplicación Web no cifra el tráfico de red correctamente cuando es necesario proteger datos y comunicaciones sensibles.

j) Fallo al restringir el acceso a URLs: la aplicación Web sólo protege la funcionalidad sensible no mostrando enlaces o URLs a usuarios no autorizados. Un atacante puede usar esta debilidad para acceder directamente de forma no autorizada a esas URLs.”

Para conocer los incidentes de mayor relevancia, se puede consultar la OWASP WHID, Web Hacking Incidents Database, de WASC ([The Web Application Security Consortium / Web-Hacking-Incident-Database \(webappsec.org\)](https://www.wasc.org/OWASP-Web-Hacking-Incident-Database)). Esta base de datos recoge los detalles sobre incidentes y ataques reales sobre entornos y aplicaciones Web, e identifica cada uno de ellos de forma unívoca con un código que refleja el año en el que se produjo el incidente, y el número de incidente dentro de ese año, como por ejemplo, “WHID 2007-82”. Otra referencia similar, para vulnerabilidades de XSS (Cross-Site Scripting), es The Daily Swig.

Desde el punto de vista de las vulnerabilidades de seguridad, diariamente se publican numerosos fallos de seguridad en productos y aplicaciones Web. Las fuentes recomendadas con información detallada respecto a vulnerabilidades de seguridad son OWASP, SANS, CCN-CERT o INCIBE.

2.3 Riesgos y amenazas de seguridad en entornos web según OWASP

La Fundación OWASP (Open Web Application Security Project) es un organismo sin ánimo de lucro formado por empresas, organizaciones educativas y particulares de todo el mundo. Es una comunidad

de seguridad informática que trabaja para crear artículos, metodologías, documentación y herramientas que se liberan y pueden ser usadas gratuitamente.

Cada tres años, publica y actualiza el proyecto **OWASP Top 10** donde se recogen los diez riesgos de seguridad más importantes en aplicaciones Web. Su última versión, de 2021 recoge los siguientes junto con su evolución desde la anterior, en 2017:

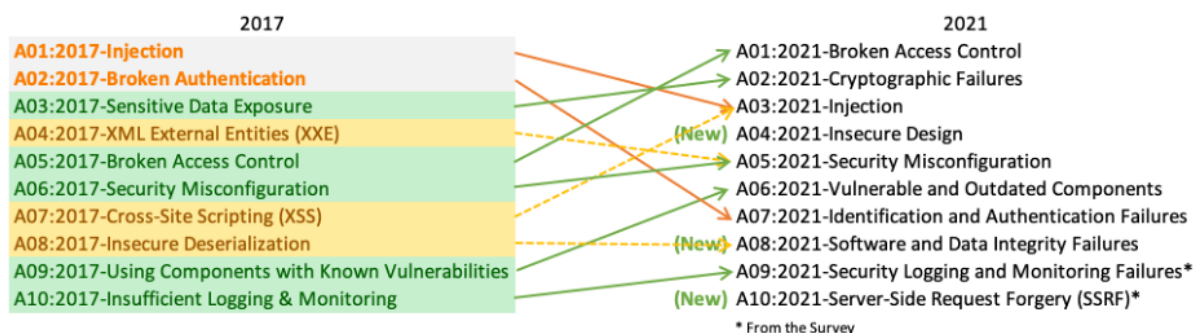


Ilustración 1: OWASP Top 10 riesgos de seguridad. Fuente: <https://owasp.org/www-project-top-ten/>

A01:2021- Broken Access control. El 94% de las aplicaciones en busca de algún tipo de control de acceso roto/vulnerable.

Esta amenaza tiene que ver con la interfaz de usuario de las aplicaciones web. El control de acceso hace cumplir políticas de manera que los usuarios no puedan actuar fuera de sus permisos previstos. Los fallos normalmente dan lugar a la divulgación no autorizada de información, modificación o destrucción de datos, o a la realización de acciones fuera de los límites del usuario. Las vulnerabilidades comunes de control de acceso incluyen:

- Violación del principio de menor privilegio o denegación por defecto, donde el acceso solo debería ser otorgado para capacidades, roles o usuarios particulares, pero está disponible para cualquiera.
- Bypass de controles de acceso mediante la modificación de la URL (manipulación de parámetros o navegación forzada), estado interno de la aplicación o la página HTML, o mediante el uso de una herramienta de ataque que modifica las solicitudes de API.
- Permitir ver o editar la cuenta de otra persona al proporcionar su identificador único (referencias directas inseguras a objetos).
- Acceder a una API sin controles de acceso para las operaciones POST, PUT y DELETE.
- Elevación de privilegios. Actuar como un usuario sin haber iniciado sesión o actuar como un administrador estando registrado como usuario.
- Manipulación de metadatos, como repetir o modificar un token de control de acceso JSON Web Token (JWT), o una cookie o campo oculto manipulado para elevar privilegios o abusar de la invalidación de JWT.
- Configuración incorrecta de CORS que permite el acceso no autorizado o de orígenes no confiables a la API.
- Navegación forzada a páginas autenticadas como usuario no autenticado o a páginas privilegiadas como usuario estándar.

A02:2021-Fallos Criptográficos. Los fallos relacionados con la criptografía a menudo conducen a la exposición de datos sensibles o al compromiso del sistema.

A03:2021-Inyección. El 94% de las aplicaciones en busca de algún tipo de inyección. El Cross-site Scripting forma parte de esta categoría.

A04:2021-Diseño Inseguro. Se recomienda un mayor uso de la modelización de amenazas, patrones y principios de diseño seguros, y arquitecturas de referencia.

A05:2021-Configuración de Seguridad Incorrecta. El 90% de las aplicaciones en busca de algún tipo de configuración incorrecta. Con el crecimiento del software altamente configurable, esta categoría sube.

A06:2021-Componentes Vulnerables y Obsoletos. Es necesario evaluar el riesgo y gestionarlo de forma continua.

A07:2021-Fallos en la Identificación y Autenticación, se ha detectado que la mayor disponibilidad de marcos estandarizados está mejorando este riesgo.

A08:2021-Fallos en la Integridad de Datos y Software es una nueva categoría para 2021 que se centra en hacer suposiciones relacionadas con las actualizaciones de software, datos críticos y canalizaciones de CI/CD sin verificar la integridad.

A09:2021-Fallos en el Registro y Monitoreo de Seguridad. Es difícil de probar y cuantificar, sin embargo, los fallos en esta categoría pueden impactar directamente en la visibilidad, la alerta de incidentes y la investigación forense.

A10:2021-Falsificación de Peticiones del Lado del Servidor. Los fallos de SSRF (Server-Side Request Forgery) ocurren siempre que una aplicación web está obteniendo un recurso remoto sin validar la URL proporcionada por el usuario. Esto permite a un atacante forzar a la aplicación a enviar una solicitud manipulada a un destino inesperado, incluso cuando está protegido por un firewall, una VPN o otro tipo de lista de control de acceso en la red (ACL, por sus siglas en inglés). A medida que las aplicaciones web modernas brindan a los usuarios finales características convenientes, la obtención de una URL se convierte en un escenario común. Como resultado, la incidencia de SSRF está aumentando. Además, la gravedad de SSRF está aumentando debido a los servicios en la nube y a la complejidad de las arquitecturas.

Se puede consultar un listado completo de los ataques existentes a través de la página web [Attacks | OWASP Foundation](#).

2.4 Conclusiones

Tanto las aplicaciones web como el entorno web cuentan con vulnerabilidades explotables dadas sus características intrínsecas, lo cual supone un riesgo para la seguridad que los responsables deben gestionar a lo largo de todo su ciclo de vida.

3. Estrategia y metodología de seguridad de aplicaciones web. Protección de entornos web.

3.1 Introducción

La estrategia y el enfoque de seguridad para aplicaciones web consisten en múltiples componentes que se complementan mutuamente. En esta sección, se describen los aspectos esenciales que deben considerarse en el diseño, desarrollo, mantenimiento y evaluación de la seguridad en entornos web.

3.2 Seguridad en entornos Web

Los elementos clave de seguridad en un entorno o aplicación Web deben incluir:

- a) Formación en seguridad de aplicaciones web.
- b) Una arquitectura e infraestructura segura para sistemas y redes.
- c) Metodología de seguridad en el desarrollo de aplicaciones web.
- d) Metodología de análisis de seguridad en aplicaciones web.

La estrategia y metodología de seguridad debe incluir adicionalmente los siguientes componentes:

- a) Formación en seguridad
- b) Instalación y configuración segura de sistemas y redes (arquitectura)
 - Actualizaciones: servidor Web y de aplicación, framework, etc
- c) Desarrollo de software seguro
 - Gestión de versiones y actualizaciones
- d) Web Application Firewalls (WAF)
- e) Auditorías de seguridad
 - Caja negra: pruebas de intrusión y Web Application Security Scanners (WASS)
 - Caja blanca: revisión de código manual y automático
- f) Respuesta ante incidentes

Para lograr un nivel adecuado de seguridad en un entorno o aplicación web, es esencial involucrar tanto a administradores como a desarrolladores. No es posible garantizar la seguridad de un entorno web sin la colaboración de ambos grupos.

La capacitación en seguridad debe centrarse en proporcionar un conocimiento sólido a los administradores y desarrolladores sobre las vulnerabilidades y amenazas en entornos web, los diferentes tipos de ataques existentes y los mecanismos de defensa asociados, preferiblemente a través de ejemplos prácticos, que les permitan construir infraestructuras y aplicaciones web más seguras.

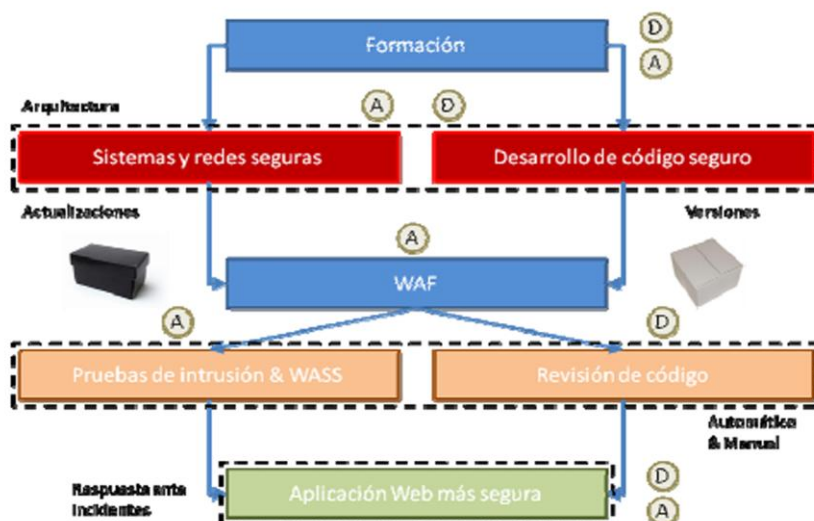


Ilustración 1. Estrategia y metodología de seguridad de aplicaciones web

Nota: dentro de las diferentes fases de la estrategia de seguridad de aplicaciones Web, algunas tareas están más asociadas a los administradores (A) y otras a los desarrolladores (D). Los círculos de la figura que contienen las letras A y D denotan el grupo principal al que se asocia cada tarea.

Ilustración 2: Estrategia y metodología de seguridad de aplicaciones web. Fuente: Guía CCN-STIC 812

3.3 Arquitecturas de seguridad en entornos web

Existen principalmente dos modelos de arquitectura para aplicaciones web:

- Modelo de dos capas, en el que el servidor web y de aplicaciones coexisten en el mismo sistema.
- Modelo de tres capas, en el que cada elemento (servidor web, servidor de aplicaciones y servidor de base de datos) es un sistema independiente.

Desde una perspectiva de seguridad, el modelo de tres capas es preferible, ya que proporciona separación entre los distintos componentes y un mayor aislamiento ante incidentes de seguridad en cualquiera de los elementos.

Además, aunque es más complejo, el modelo de tres capas ofrece:

- Mayor escalabilidad para gestionar un mayor número de peticiones y usuarios en la aplicación.
- La capacidad de introducir controles de acceso avanzados, como el filtrado de tráfico, y elementos avanzados de monitoreo, como sistemas de detección de intrusiones, entre los diferentes elementos de la arquitectura.
- La posibilidad de aplicar una seguridad más estricta a cada uno de los componentes, ya que cada uno tiene funciones y tareas claramente definidas.

Al diseñar una aplicación web, es necesario tener en cuenta múltiples factores, como la arquitectura, el desarrollo de la aplicación y los mecanismos de auditoría.

La arquitectura de la aplicación web debe contar con mecanismos de detección y protección a nivel de red, incluyendo elementos de seguridad tradicionales como cortafuegos o sistemas de detección de intrusiones. Se recomienda seguir las directrices de la guía CCN para cortafuegos de seguridad perimetral: CCN-STIC-408 Seguridad Perimetral – Cortafuegos. Uno de los elementos clave en la protección de entornos y aplicaciones web son los cortafuegos de aplicaciones web (WAF Web Application Firewall), también conocidos como cortafuegos de aplicaciones web.

Es importante contar con dispositivos dedicados exclusivamente a la inspección y el filtrado del tráfico web, es decir, el protocolo HTTP (o HTTPS), ya que los cortafuegos tradicionales no disponen de capacidades avanzadas para analizar y bloquear los ataques recibidos a través de este protocolo. Los WAF suelen ubicarse, desde una perspectiva de arquitectura de red, frente al servidor web, con el objetivo de monitorizar y filtrar todas las peticiones maliciosas recibidas por la aplicación web.

En arquitecturas web de tres capas (servidor web, servidor de aplicaciones y base de datos), es posible utilizar WAF dedicados entre cada una de las capas.

Algunos de los WAF más conocidos y ampliamente utilizados, además de las soluciones comerciales, incluyen mod_security (para Apache) y URLScan/Lockdown (para Microsoft IIS).

Algunos de los WAF más conocidos en el ámbito comercial son AppShield, F5, DenyAll, SecureIIS, Microsoft ISA, Profense, Cloudflare, Imperva WAF, Forti WAF, Akamai etc.

Al evaluar un WAF, se consideran varios criterios, como el modo de operación, la gestión de SSL/TLS, las técnicas de bloqueo de tráfico, el tipo de solución (appliance o software), las capacidades de reescritura de solicitudes web, la alta disponibilidad y el rendimiento de la solución, el soporte para otros protocolos además de HTTP, la detección de ataques mediante firmas y técnicas de normalización, la protección contra ataques de fuerza bruta, cookies, sesiones, campos HTML ocultos y más.

Si se requiere confidencialidad en las comunicaciones web, se debe utilizar HTTPS, que utiliza SSL (Secure Socket Layer) o TLS (Transport Layer Security) para cifrar las comunicaciones entre el navegador web y el servidor web. Se recomienda utilizar TLS a partir de la versión 1.2 y habilitar algoritmos de cifrado robustos.

Además de los elementos de seguridad propios de un entorno web, es necesario proteger todos los componentes de la infraestructura en la que reside la aplicación web, como dispositivos de comunicación (routers, switches, etc.) o la infraestructura de servidores de nombres (DNS).

Los mecanismos de protección implementados deben proteger los diferentes equipos frente a:

- a) Ataques directos, como accesos no autorizados a cualquiera de los elementos que componen el entorno o la aplicación web.
- b) Ataques indirectos, en los que cualquiera de los elementos se utiliza como herramienta en el ataque. Por ejemplo, un ataque a los servidores de DNS podría redirigir el tráfico de los clientes web a un entorno malicioso que suplante a la aplicación web real (lo que se conoce como *pharming*).
- c) Ataques de denegación de servicio (DoS). Las arquitecturas web se basan en tecnologías TCP/IP y, por lo tanto, son vulnerables a los ataques comunes de DoS en TCP/IP.

Es fundamental aplicar las últimas actualizaciones de seguridad en todos los elementos de software que forman parte de la plataforma de la aplicación web, incluyendo software de dispositivos de red y firewalls, sistemas operativos de servidores (web, aplicación y base de datos) y software de la plataforma de desarrollo utilizada (PHP, ASP, Java, etc.).

Desde una perspectiva de infraestructura web, es necesario realizar un análisis detallado de los usuarios, grupos, permisos y derechos con los que se ejecutarán tanto cada uno de los componentes de la aplicación web, como los procesos del servidor web o de la aplicación. Este análisis permitirá aplicar todos los mecanismos de autorización necesarios. Además, es esencial contar con un análisis detallado de las credenciales con las que los componentes podrán acceder y obtener información de otros componentes.

3.4 Conclusión

Para lograr un nivel adecuado de seguridad en un entorno o aplicación web, es crucial involucrar tanto a administradores como a desarrolladores y proporcionarles un conocimiento sólido sobre las vulnerabilidades y amenazas de seguridad en entornos web, los diferentes tipos de ataques existentes y los mecanismos de defensa asociados. Esto permitirá construir una arquitectura de seguridad adecuada que garantice una infraestructura y aplicaciones web más seguras.

4. Desarrollo seguro del software de aplicaciones

4.1 Introducción

El componente esencial dentro de la estrategia de seguridad aplicada al desarrollo de software para aplicaciones web implica abordar todos los aspectos relacionados con la seguridad a lo largo del ciclo de vida de desarrollo de software (SDLC, por sus siglas en inglés, Software Development Life Cycle).

Generalmente, las aplicaciones web no están sometidas a un riguroso sistema de control de versiones que permita mantener un seguimiento constante de la versión en producción, como ocurre con otros tipos de software.

Debido a la naturaleza dinámica y a la constante demanda de nuevas funcionalidades en este tipo de aplicaciones, se incorpora nuevo código y contenido a la aplicación web en tiempo real, sin llevar a cabo un control minucioso de los cambios realizados.

4.2 Recomendaciones generales

En términos generales, se aconseja utilizar el método GET de HTTP únicamente para la consulta de información, mientras que el método POST debe emplearse para la transferencia y envío de datos desde los clientes web hacia la aplicación web.

Es importante destacar que las cabeceras HTTP pueden ser manipuladas con relativa facilidad por un posible atacante, por lo que no deben ser utilizadas como medio de validación o envío de información crítica.

Todas las interacciones entre los distintos componentes del entorno web, como el servidor web, el servidor de aplicaciones y la base de datos, deben llevarse a cabo de manera segura. Esto implica que las comunicaciones entre estos elementos deben ser encriptadas, autenticadas y asegurar la integridad de los datos transmitidos.

Asimismo, cualquier información sensible, ya sea parte de la lógica de la aplicación o credenciales de acceso, debe almacenarse de forma cifrada en todos los servidores, con especial atención al servidor de la base de datos.

4.2.1 Filtrado de datos de entrada del usuario

Es esencial mantener una consistencia en la aplicación de medidas de seguridad, como el filtrado de datos proporcionados por los usuarios. Algunos de los ataques web mencionados, como el XSS o la inyección SQL, pueden ser contrarrestados filtrando los datos maliciosos (como código HTML y scripts, o sentencias SQL) en la entrada de la aplicación, en la salida, o en ambos puntos. Siguiendo el principio de defensa en profundidad, se recomienda aplicar los mecanismos de filtrado tanto en la entrada como en la salida de datos. Es fundamental considerar que los datos de entrada proporcionados por usuarios (o posibles atacantes) deben ser tratados como potencialmente dañinos por naturaleza, por lo que es necesario verificar y analizar estos datos antes de procesarlos en la aplicación.

Existen dos enfoques para el filtrado de datos de entrada:

- a) Eliminar los caracteres maliciosos y permitir el resto.
- b) Permitir únicamente los caracteres válidos para cada tipo de entrada en la aplicación.

Desde una perspectiva de seguridad, el segundo enfoque es más riguroso y se recomienda, aunque en ocasiones puede no ser factible debido a la falta de conocimiento sobre el conjunto completo de caracteres válidos.

Se aconseja establecer una única biblioteca de código que abarque todas las funciones necesarias para filtrar cualquier dato de entrada proporcionado por el usuario. Cada entrada recibida por parte del usuario debe ser procesada y filtrada por esta biblioteca, independientemente del método utilizado, ya sea a través de variables en métodos GET o POST, cookies, cabeceras HTTP (como Referer o User-Agent), entre otros.

La disponibilidad de una única biblioteca central facilita la aplicación consistente de los mismos mecanismos de filtrado en cualquier parte del código de la aplicación. Para lograrlo, solo es necesario invocar la función correspondiente de la biblioteca desde el código. Además, la existencia de una única biblioteca central simplifica el proceso de mantenimiento y actualización de los mecanismos de filtrado en toda la aplicación web.

4.2.2 Mensajes de error y otros contenidos

El entorno web debe implementar una gestión de errores apropiada con el objetivo de reducir al mínimo la cantidad de información que se divulga al usuario o a un posible atacante. La información contenida en los mensajes de error puede ser utilizada durante la fase de reconocimiento del entorno, lo que podría proporcionar detalles sobre el software y sus versiones, información del sistema de archivos y pistas sobre la ubicación de recursos utilizados por la aplicación web. Además, estos mensajes podrían incluir información más específica sobre la base de datos, lo que podría ser explotado por un atacante para llevar a cabo ataques de inyección de código más efectivos.

Por lo tanto, es esencial que en todo momento se capturen las condiciones de error y se muestren mensajes de error personalizados que divulguen la menor cantidad de información posible. Se desaconseja mostrar directamente los mensajes de error detallados generados por el servidor web, el servidor de aplicaciones o la base de datos.

4.2.3 Autenticación y gestión de sesiones

Un ejemplo específico de mensajes de error se relaciona con el proceso de autenticación. En ningún caso la aplicación debe revelar información sobre qué componentes de las credenciales de acceso no son válidos. Por ejemplo, si se emiten mensajes de error que indican que el usuario es válido, pero la contraseña no lo es, esto facilita en gran medida a un atacante la tarea de adivinar contraseñas.

El mecanismo de autenticación debe incluir características que permitan definir políticas de acceso, determinar la longitud y complejidad de las contraseñas, establecer mecanismos de registro de intentos de acceso, y definir una política de bloqueo temporal de cuentas después de un cierto número de intentos fallidos.

En lo que respecta a la gestión de sesiones en la aplicación web, es importante utilizar identificadores de sesión o tokens que sean impredecibles (es decir, suficientemente aleatorios) y tengan una longitud adecuada para que no puedan ser descifrados mediante ataques de fuerza bruta. Además, estos identificadores deben tener una fecha de vencimiento, lo que significa que expiran después de cierto período de tiempo.

4.2.4 CSRF

Los ataques CSRF (Cross-Site Request Forgery) solo pueden prevenirse mediante la implementación de elementos conocidos como "tokens de formulario" o "tokens anti-CSRF". La aplicación debe ser capaz de verificar su propia lógica antes de llevar a cabo una acción crítica. Esto se logra mediante el uso de tokens dinámicos, que son generados para cada sesión de usuario y para cada formulario crítico.

Estos tokens deben ser lo suficientemente aleatorios y deben tener una fecha o un tiempo de expiración.

Otras soluciones, como basar la validez de las solicitudes en el campo "Referer" (cabecera HTTP), no son efectivas, ya que este campo puede ser fácilmente modificado por un atacante.

Además, es posible prevenir este tipo de ataques y brindar un mayor control sobre la aplicación web al usuario mediante la solicitud de verificación para acciones críticas. Una implementación común de esta verificación es el uso de imágenes CAPTCHA, donde el usuario debe ingresar el texto presente en una imagen, o la utilización de la autenticación de dos factores mediante tarjetas de coordenadas o dispositivos de claves de un solo uso.

4.2.5 Gestión de logs

Existen múltiples razones para la generación, almacenamiento y gestión de registros o logs en una aplicación web, que van desde requisitos legales y regulatorios hasta necesidades de contabilidad, solución de problemas, auditoría, recopilación de estadísticas, respuesta ante incidentes y análisis forense.

Se recomienda encarecidamente seguir las mejores prácticas en la gestión de registros o logs, no solo a nivel de los diversos componentes que conforman la infraestructura de la aplicación web, como los dispositivos de red, cortafuegos, sistemas de detección de intrusiones, cortafuegos de aplicaciones web, servidor web, servidor de aplicaciones o servidor de base de datos, sino también en el propio código de la aplicación web.

4.3 Conclusión

Se detallan las recomendaciones específicas de seguridad para aplicaciones o entornos Web, dado que su desarrollo suele ser dinámico y suele requerir la incorporación de nuevas funcionalidades a lo largo de su ciclo de vida, que debe realizarse de manera segura.

5. Análisis de seguridad de aplicaciones web

5.1 Introducción

Una vez se ha implementado una metodología de desarrollo de software para aplicaciones Web que contempla los aspectos de seguridad pertinentes, es necesario complementarla con análisis y auditorías del entorno y la aplicación Web.

El objetivo del análisis de seguridad de una aplicación Web es pasar de una aplicación Web que presenta vulnerabilidades, a una aplicación que es segura y no tiene vulnerabilidades conocidas.

Es importante reseñar que la seguridad al 100% no se puede asegurar, por lo que el objetivo de la metodología de análisis es aumentar lo más posible el nivel de seguridad de la aplicación Web mediante el estudio de aquellas vulnerabilidades y errores conocidos.

La metodología de análisis debe incluir dos áreas (descritas en detalle a continuación):

- a) Caja negra
- b) Caja blanca

5.2 Metodología de análisis de caja negra

El análisis de caja negra se centra en estudiar las vulnerabilidades de seguridad de la aplicación Web desde el punto de vista de un atacante externo.

El analista, actuando como atacante, no dispone de ningún tipo de información previa relativa a la aplicación y mucho menos del código de la misma.

El análisis se basa en la interacción del analista con la aplicación Web. Mediante la generación de diferentes estímulos o datos de entrada, se analiza la respuesta o datos de salida, con el objetivo de identificar posibles vulnerabilidades.

5.3 Metodología de análisis de caja blanca

El análisis de caja blanca se centra en estudiar las vulnerabilidades de seguridad de la aplicación Web desde el punto de vista del desarrollador.

El analista, actuando como desarrollador, dispone de acceso completo al código fuente de la aplicación para su revisión.

La metodología a seguir se centra en realizar un análisis exhaustivo del código de la aplicación en busca de funciones vulnerables o de la ausencia de métodos que permitan, por ejemplo, validar la entrada recibida por el usuario.

Desde un punto de vista general, a la hora de establecer controles específicos sobre una aplicación Web es necesario conocer las recomendaciones y soluciones disponibles para el lenguaje de programación empleado: Java, PHP, .NET, etc. Por este motivo, es necesario disponer de documentación adicional relativa al lenguaje de programación de la aplicación Web, tal como funciones y librerías que permiten validar y normalizar las entradas del usuario en el lenguaje empleado.

5.4 Conclusión

Es necesario complementar una metodología de desarrollo de software para aplicaciones Web que contempla los aspectos de seguridad pertinentes, con análisis y auditorías del entorno y la aplicación Web para detectar vulnerabilidades y solventarlas, consiguiendo una aplicación segura, sin vulnerabilidades conocidas.

6. RESUMEN ESQUEMÁTICO

1. LA SEGURIDAD EN EL NIVEL DE APLICACIÓN.
2. TIPOS DE ATAQUES DE APLICACIONES Y ENTORNOS WEB.
 - 2.1 INTRODUCCIÓN
 - 2.2 RIESGOS Y AMENAZAS DE SEGURIDAD EN ENTORNOS WEB
 - 2.3 RIESGOS Y AMENAZAS DE SEGURIDAD EN ENTORNOS WEB SEGÚN OWASP
 - 2.4 CONCLUSIONES
3. ESTRATEGIA Y METODOLOGÍA DE SEGURIDAD DE APLICACIONES WEB. PROTECCIÓN DE ENTORNOS WEB.
 - 3.1 INTRODUCCIÓN
 - 3.2 SEGURIDAD EN ENTORNOS WEB
 - 3.3 ARQUITECTURAS DE SEGURIDAD EN ENTORNOS WEB
 - 3.4 CONCLUSIÓN
4. DESARROLLO SEGURO DEL SOFTWARE DE APLICACIONES
 - 4.1 INTRODUCCIÓN
 - 4.2 RECOMENDACIONES GENERALES
 - 4.2.1 FILTRADO DE DATOS DE ENTRADA DEL USUARIO
 - 4.2.2 MENSAJES DE ERROR Y OTROS CONTENIDOS
 - 4.2.3 AUTENTIFICACIÓN Y GESTIÓN DE SESIONES
 - 4.2.4 CSRF
 - 4.2.5 GESTIÓN DE LOGS
 - 4.3 CONCLUSIÓN
5. ANÁLISIS DE SEGURIDAD DE APLICACIONES WEB
 - 5.1 INTRODUCCIÓN
 - 5.2 METODOLOGÍA DE ANÁLISIS DE CAJA NEGRA
 - 5.3 METODOLOGÍA DE ANÁLISIS DE CAJA BLANCA
 - 5.4 CONCLUSIÓN

7. GLOSARIO

- BD Base de datos
- CAPTCHA Completely Automated Public Turing test to tell Computers and Humans Apart
- CCN Centro Criptológico Nacional
- CCN-CERT Centro Criptológico Nacional – Computer Emergency Response Team
- CSRF Cross-Site Request Forgery
- DDoS Distributed Denial of Service
- DNS Domain Name Service (o System)
- DoS Denial of Service
- HTML Hyper Text Markup Language
- HTTP Hyper Text Transfer Protocol
- HTTPS Secure Hyper Text Transfer Protocol
- IDS Intrusion Detection System
- IIS Internet Information Server
- LDAP Lightweight Directory Access Protocol
- OWASP Open Web Application Security Project
- PCAP Packet Capture
- RFI Remote File Inclusion
- SDLC Software Development Life Cycle
- SQL Structured Query Language
- SSL Secure Socket Layer
- STIC Seguridad de las Tecnologías de la Información y las Comunicaciones
- TIC Tecnologías de la Información y las Comunicaciones
- TLS Transport Layer Security
- URI Uniform Resource Identifier
- URL Uniform Resource Locator
- VNC Virtual Network Computing
- VoIP Voz sobre IP
- WAF Web Application Firewall
- WASC Web Application Security Consortium
- WASS Web Application Security Scanner
- Webmail Correo electrónico accesible vía Web
- WHID Web Hacking Incidents Database
- XML eXtended Markup Language
- XSS Cross-Site Scripting

8. TEST

a) PREGUNTAS DE TEST

- 1) ¿En qué fases se debe cumplir con los requisitos de seguridad de un entorno o aplicación web?
 - a) Durante el proceso de diseño y desarrollo de la aplicación, estableciendo unos requisitos y verificaciones de seguridad que debe cumplir toda aplicación Web.
 - b) Posteriormente, tras la puesta en producción, realizando el análisis de la aplicación Web mediante auditorías de seguridad, análisis de vulnerabilidades y pruebas de intrusión.
 - c) Las dos anteriores
 - d) Durante el proceso de diseño y desarrollo de la aplicación, puesto que el desarrollo también se da mientras se encuentra en producción.
- 2) Las aplicaciones Web estándar utilizan el puerto TCP/80 (HTTP) y las que emplean cifrado mediante SSL (Secure Socker Layer) o TLS (Transport Layer Security) emplean el puerto TCP/443 (HTTPS). Por tanto:
 - a) Los firewalls tradicionales de manera general deben dejar pasar el tráfico hacia estos puertos, y son de gran utilidad en el filtrado de ataques directos sobre la aplicación Web.
 - b) Los firewalls tradicionales de manera general no deben dejar pasar el tráfico hacia estos puertos, y son de gran utilidad en el filtrado de ataques directos sobre la aplicación Web.
 - c) Los firewalls tradicionales de manera general no deben dejar pasar el tráfico hacia estos puertos, y son de poca utilidad en el filtrado de ataques directos sobre la aplicación Web.
 - d) Los firewalls tradicionales de manera general deben dejar pasar el tráfico hacia estos puertos, y son de poca utilidad en el filtrado de ataques directos sobre la aplicación Web.
- 3) Según el artículo 33 del RD 311/2022, el organismo que articulará la respuesta a los incidentes de seguridad y que actuará sin perjuicio de las capacidades de respuesta a incidentes de seguridad que pueda tener cada administración pública:
 - a) INCIBE-CERT.
 - b) CCN.
 - c) SOC de la AGE.
 - d) ENISA.
- 4) Uno de los elementos principales empleados en la protección de entornos y aplicaciones Web son:
 - a) Los cortafuegos (firewalls) de aplicaciones Web, también conocidos como WAF, Web Application Firewall
 - b) Los DNS (Servidores de Nombre de Dominio)
 - c) Los CSRF (Cross-Site Request Forgery)
 - d) Los WASS (Web Application Security Socket)
- 5) Algunos de los ataques Web mencionados, como por ejemplo XSS o inyección SQL, pueden ser mitigados filtrando los datos maliciosos (código HTML y scripts, o sentencias SQL, respectivamente):
 - a) en la entrada de la aplicación
 - b) en la salida de la aplicación
 - c) se recomienda en ambas
 - d) en la entrada de la aplicación si hay formularios, puesto que es lo más común conforme al principio de proporcionalidad

b) SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	c
2	d
3	b
4	a
5	c

9. BIBLIOGRAFÍA

a. BIBLIOGRAFÍA BÁSICA

1. [812-Entornos y aplicaciones web.pdf \(cni.es\)](#) / Guía de Seguridad de las TIC CCN-STIC-812. GUÍA DE SEGURIDAD EN ENTORNOS Y APLICACIONES WEB.

b. BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

1. <https://www.owasp.org> / Open Web Application Security Project (OWASP).
2. <https://www.sans.org/> / SANS Software Security Institute (ésta cubre no sólo el desarrollo seguro de aplicaciones Web, sino también el desarrollo de cualquier otro tipo de aplicaciones software)